

第三模块：RTOS上下文切换与调度机制（内核级超详细版 • Cortex-M 方向）

本模块不讲API，不讲如何创建任务。

目标只有一个：

★ 解释 RTOS 任务切换时 CPU 到底发生了什么。

本模块基于 Cortex-M 异常模型，属于真正底层执行机制。

一、RTOS 为什么一定依赖异常机制

RTOS 不能自己暂停当前任务。

原因：

CPU 只有在 **异常（Exception）** 中才允许改变执行上下文。

因此：

👉 RTOS 必须借助 PendSV。

二、PendSV 为什么被设计成最低优先级

核心思想：

调度不是最高优先级行为。

如果 PendSV 优先级过高：

- 会打断实时中断
- 导致控制任务抖动

因此：

RTOS 将 PendSV 设置为最低优先级。

三、任务切换真实流程（CPU级行为）

一次任务切换不是函数调用，而是：

👉 CPU上下文搬运。

流程：

1) SysTick 或中断触发调度请求 2) 设置 PendSV 挂起位 3) 进入 PendSV_Handler 4) 保存当前任务寄存器 5) 切换任务栈指针 6) 恢复新任务寄存器

四、上下文保存为什么只保存 R4-R11（核心重点）

异常入口已经自动保存：

R0 R1 R2 R3 R12 LR PC xPSR

因此 RTOS 只需保存：

R4-R11

这就是 FreeRTOS 汇编代码看起来“少保存寄存器”的原因。

五、PendSV 汇编流程（核心理解）

典型流程（伪汇编解释）：

```
MRS R0, PSP      ; 读取当前任务栈
STMDB R0!, {R4-R11}; 保存寄存器
STR R0, [TCB]     ; 保存栈指针
```

然后：

```
LDR R0, [NextTCB]
LDMIA R0!, {R4-R11}
MSR PSP, R0
```

最后：

BX LR

CPU 自动恢复异常栈帧。

六、BASEPRI 在 RTOS 中的真正作用

RTOS 并不会完全关闭中断。

而是：

👉 提升 BASEPRI 屏蔽低优先级中断。

好处：

- 高优先级中断仍可运行
 - 保证调度代码原子性
-

七、为什么 FreeRTOS 不能在任意地方切换任务

原因：

只有在异常上下文中，CPU 才允许安全切换栈。

因此任务切换必须通过：

- PendSV
 - SVC
-

八、SysTick 在调度中的角色

SysTick 只做一件事：

更新 tick 计数并决定是否触发调度。

真正切换任务的不是 SysTick，而是 PendSV。

九、任务栈真实结构（很多人没见过）

创建任务时，RTOS 会伪造一个异常栈帧：

```
PC = 任务入口函数
LR = 任务退出处理
xPSR = 初始状态
```

因此任务第一次运行时看起来像“从异常返回”。

十、高频底层考点（标准答案）

Q 1：为什么任务切换要用 PendSV?

标准答案：

因为 PendSV 可以被设置为最低优先级，从而避免打断实时中断。

Q 2：为什么只保存 R 4 -R 11？

标准答案：

异常入口已自动保存 R 0 -R 3 等寄存器。

Q 3：BASEPRI 与 PRIMASK 区别？

标准答案：

BASEPRI 只屏蔽低优先级中断，而 PRIMASK 关闭所有可屏蔽中断。

Q 4：任务第一次运行为什么像从中断返回？

标准答案：

因为 RTOS 在栈中构造了一个假的异常栈帧。

十一、工程注意事项（真正经验）

- 1) 不要在临界区调用阻塞API。
 - 2) PendSV 优先级必须最低。
 - 3) 理解异常栈帧才能调试RTOS问题。
-

十二、深度练习（强烈建议）

- 打开 FreeRTOS port.c 阅读 PendSV 汇编。
 - 打印 PSP 查看任务切换前后变化。
 - 手画一次任务切换栈变化图。
-

完成本模块后，你将真正理解：RTOS 调度不是软件技巧，而是 Cortex-M 异常机制的应用。